



EliteCarz- A MongoDB driven Web Application for Car Rental Management System

ADB Lab Final Project Report

Submitted to :

Mr. Qaiser Farooq

Submitted by :

**LAIBA LARAIB (SP24-BCS-053)
MUHAMMAD USAMA (SP24-BCS-078)**

(Group no. 11)

Date of Submission

May 11, 2026

COMSATS UNIVERSITY

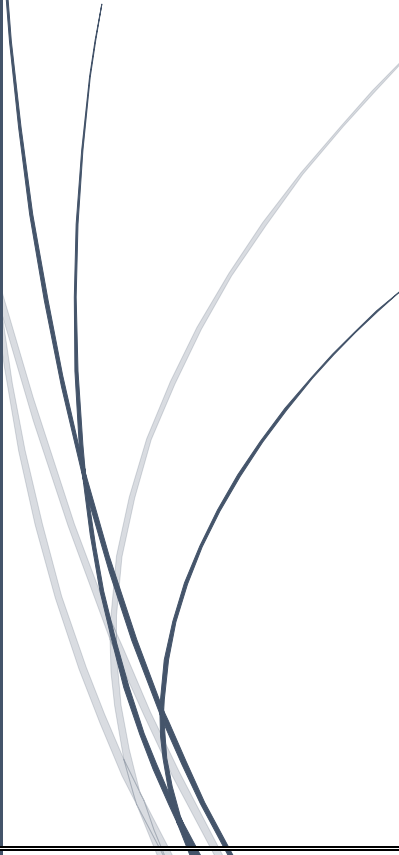


Table of Contents

1. Problem Statement	4
2. Problem Solution/ Objective of the Proposed System	5
3. Related System Analysis.....	5
4. Vision Statement	6
5. Scope.....	6
a) Admin Functionality	6
b) User Functionality	6
c) Guest Functionality	7
d) Backend Features.....	7
6. Modules.....	7
Module 1: Authentication & Authorization Module.....	7
Module 2: Admin Dashboard Module	8
Module 3: Car Management Module	9
Module 4: Booking Management Module.....	9
Module 5: Payment Management Module	10
Module 6: Reviews & Ratings Module	11
Module 7: Analytics & Reporting Module.....	12
Module 8: Contact & Content Management Module	13
7. System Limitations	13
8. Tools and Technologies	14
9. Project stakeholders and Roles	14
10. GUI Mockups.....	14
a) Login Page.....	15
b) Admins View.....	16
c) Customer View	22
11. Dataset Schema (MongoDB Collections)	25
12. Improvements made in Dataset Schema.....	30
a) Improvements Made in the Dataset Schema	30

b)	Summary of Improvements	31
i.	Users Collection	31
ii.	Cars Collection	31
iii.	Bookings Collection.....	32
iv.	Payments Collection	32
v.	Reviews Collection	33
c)	Addition of Multiple New Collections	33
13.	ER Diagrams	34
14.	Lab Implementations (Major Section)	35
14.1	CRUD Operations (LAB 1–2)	35
14.2	Aggregation Pipelines (LAB 3–4)	36
14.3	Data Consistency (Versioning + Atomic Updates).....	38
14.4	Transactions (Atomic Multi-Document Operations)	38
14.5	Concurrency Control	39
14.6	Locking Types Used.....	40
14.7	Views and Materialized Views	40
14.8	Database Security (Authentication + Access Control).....	41
14.9	NoSQL Injection Prevention.....	42
14.10	Salted Password Hashing	42
14.11	Access Control (Admin/User/Guest)	43
14.12	Use of MongoDB Atlas	43
15.	System Deployment	43
16.	Conclusion	44
17.	References	44

Abstract

This project presents the design and implementation of a MongoDB-based Car Rental Management System developed using Flask and MongoDB Atlas. The system automates car rental operations including vehicle management, customer bookings, payments, reviews, and rental tracking. The project implements major Advanced Database Systems concepts such as CRUD operations, aggregation pipelines, schema validation, transactions, concurrency control, indexing, role-based access control, and NoSQL security mechanisms.

The system provides separate interfaces for Admins and Customers where users can browse cars, create bookings, manage payments, and view rental history. MongoDB's flexible document-oriented architecture allows efficient handling of semi-structured rental data while maintaining scalability and performance. Additional features such as analytical dashboards, materialized views, optimistic locking, and salted password hashing demonstrate practical implementation of modern NoSQL database techniques.

The completed project shows how advanced database technologies can be used to build scalable real-world rental management applications.

EliteCarz- A MongoDB driven Web Application for Car Rental Management System

1. Problem Statement

Car rental companies handle large volumes of semi-structured data such as customer information, vehicle details, bookings, payments, reviews, rental history, and availability status. Traditional database systems often struggle with:

- Scalability
- flexible schema handling
- real-time analytics.
- Rigid schemas unsuited for frequently changing media structures
- Performance bottlenecks
- Limited ability to handle nested JSON-like documents

The main problem this project aims to solve is to design and implement a NoSQL-based Car Rental Management System that supports:

- CRUD operations across collections
- Customer and vehicle management
- Real-time booking operations
- Aggregation-based analytics dashboards
- Transactions and concurrency control
- Secure authentication and role-based access
- Schema validation and indexing
- MongoDB Atlas cloud integration

The system also demonstrates how Advanced Database concepts can be applied in a real-world business environment

2. Problem Solution/ Objective of the Proposed System

Main Objectives

The main objectives of our proposed system are to

- Develop a complete Car Rental Management System using MongoDB Atlas
- Implement CRUD operations for cars, bookings, users, and payments
- Build aggregation pipelines for analytics dashboards
- Implement concurrency control during booking operations
- Demonstrate transactions for booking + payment processing
- Implement schema validation using MongoDB
- Provide role-based access control (Admin/User)
- Prevent NoSQL injection attacks
- Store passwords using salted hashing
- Build a user-friendly GUI dashboard
- Use indexes and views for performance optimization

3. Related System Analysis

Modern car rental and ride-sharing platforms such as **Uber Rentals**, and **Careem** rely heavily on scalable database systems to manage vehicle inventory, customer bookings, payments, and real-time analytics.

Our system extends these ideas by implementing:

- Real-time CRUD operations
- User-specific dashboards
- Aggregation pipelines for analytics
- Secure access control (Admin/User/Guest)
- Concurrency protection
- Transactions and atomic writes
- JSON Schema validation for flexible content

Thus, the project demonstrates core concepts used by real-world rental platforms on a smaller academic scale.

4. Vision Statement

“To develop a scalable and analytics-driven Car Rental Management System named using MongoDB and Advanced Database concepts including aggregation pipelines, transactions, concurrency control, schema validation, views, and role-based security to improve rental management efficiency and user experience.”

5. Scope

The project supports the following :

a) Admin Functionality

- Manage car inventory (Add, Update, Delete cars)
- Manage customer accounts
- View and manage bookings
- Track payments and revenue
- Manage customer reviews
- Handle customer contact queries
- Manage blogs and website content
- Access analytical dashboards and reports

b) User Functionality

- Register and login securely
- Browse available cars
- Search and filter cars
- Create rental bookings
- View booking history
- Submit reviews and ratings
- Contact administrators

- View rental and payment details

c) Guest Functionality

- Browse available cars
- View car details and rental prices
- Read blogs and website content
- Submit contact queries

d) Backend Features

- CRUD operations using MongoDB
- Aggregation pipelines for analytics
- Transactions for booking and payment processing
- Concurrency control using locking and versioning
- MongoDB Views and Materialized Views
- Role-Based Access Control (RBAC)
- Password hashing and authentication security
- MongoDB Atlas cloud database integration

6. Modules

Module 1: Authentication & Authorization Module

This module manages user authentication, registration, session handling, and role-based access control within the EliteCarz platform.

Features

- User Registration
- Secure Login System
- Session Management
- Password Hashing
- Role-Based Access Control (Admin/User/Guest)

Technologies Used

- Flask Sessions
- bcrypt Password Hashing
- MongoDB Users Collection

Code Snippet

```
hashed_password = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
```

Module 2: Admin Dashboard Module

The Admin Dashboard provides centralized control over the EliteCarz platform and displays analytical insights related to bookings, revenue, users, and cars.

Features

- Total Cars Statistics
- Total Bookings Management
- Revenue Analytics
- Queries management
- Top Rented Cars Analytics
- User Management
- Dynamic CRUD Operations
- Reviews Management

Collections Used

- bookings
- cars
- users
- payments
- mv_monthly_revenue
- mv_top_cars_current_month

Code Snippet

```
db.bookings.aggregate([
  {
    "$group": {
      "_id": None,
      "total_revenue": {"$sum": "$total_price"}
    }
  }
])
```

Module 3: Car Management Module

This module handles all operations related to rental vehicles available on the EliteCarz platform.

Features

- Add New Cars
- Update Car Information
- Delete Cars
- Search and Filter Cars
- Manage Car Availability
- Upload Car Images

Collections Used

- cars

Code Snippet

```
db.cars.update_one(
  {"_id": "CAR001"},
  {"$set": {"availability": False}}
)
```

Module 4: Booking Management Module

The Booking Module manages rental bookings and reservation workflows between users and vehicles.

Features

CSC 316 – Advanced Database Systems

- Create Booking
- Cancel Booking
- View Booking History
- Track Booking Status
- Prevent Double Booking
- Booking Version Control

Collections Used

- bookings
- counters
- locks

Advanced Database Features

- Optimistic Locking
- Pessimistic Locking
- Atomic Updates

Code Snippet

```
db.bookings.update_one(  
  {  
    "_id": "BKG0009",  
    "version": 1  
  },  
  {  
    "$set": {"status": "confirmed"},  
    "$inc": {"version": 1}  
  }  
)
```

Module 5: Payment Management Module

This module handles payment records and revenue-related operations for the platform.

Features

- Payment Processing

CSC 316 – Advanced Database Systems

- Payment Status Tracking
- Revenue Calculation
- Booking-Payment Relationship Management
- Monthly Revenue Analytics

Collections Used

- payments
- booking_payment_view
- mv_monthly_revenue

Code Snippet

```
db.payments.insert_one({
  "_id": "PAY0200",
  "booking_id": "BKG0200",
  "amount": 25000,
  "status": "paid"
})
```

Module 6: Reviews & Ratings Module

This module allows customers to submit reviews and ratings for rented vehicles.

Features

- Submit Reviews
- Submit Ratings
- View Customer Feedback
- Calculate Average Ratings

Collections Used

- reviews
- cars

Code Snippet

```
db.reviews.insert_one({
  "_id": "REV100",
  "user_id": "USR0009",
  "car_id": "CAR001",
  "rating": 5,
  "comment": "Excellent experience"
})
```

Module 7: Analytics & Reporting Module

This module implements aggregation pipelines, views, and materialized views to generate analytical insights for the EliteCarz platform.

Features

- Monthly Revenue Reports
- Revenue by City
- Top Performing Cars
- Booking Statistics
- Aggregation-Based Dashboards

Collections / Views Used

- mv_monthly_revenue
- mv_revenue_by_city_current_month
- mv_top_cars_current_month
- user_booking_view
- booking_payment_view

Code Snippet

```
db.bookings.aggregate([
  {
    "$group": {
      "_id": "$car_id",
      "total_bookings": {"$sum": 1}
    }
  }
])
```

Module 8: Contact & Content Management Module

This module manages website content, customer queries, and blogs.

Features

- Manage Blogs
- Manage Website Content
- Handle Contact Queries
- Admin Reply System

Collections Used

- blog_posts
- contact_queries
- site_content

Code Snippet

```
db.contact_queries.update_one(  
  {"_id": query_id},  
  {"$set": {  
    "status": "resolved",  
    "admin_reply": reply  
  }}  
)
```

7. System Limitations

- MongoDB Atlas free-tier limitations
- **No Sharding Implemented** : As per the instructor's requirements horizontal scaling through sharding were not included, limiting the system's capability to distribute data across multiple nodes
- **Limited Concurrency Simulation**: Due to resource restrictions, concurrency control demonstrations (locking, transactions, and multi-user editing were simulated at application level rather than tested at large scale.

- Limited cloud deployment

8. Tools and Technologies

The tools and technologies being used in the project are :

Tools	Usage
Python Flask	Backend API
MongoDB Atlas	Cloud NoSQL database
MongoDB Views / Aggregations	Analytics
Bootstrap 5	Responsive UI
Charts.js	Visual dashboards
JSON Schema Validations	Data Integrity
BCrypt	Secure Passwords
JavaScript	Frontend logic

9. Project stakeholders and Roles

Role	Responsibility
Admin	Administrators manage the complete EliteCarz platform including cars, bookings, payments, reviews, blogs, contact queries, and analytical dashboards. They also monitor revenue reports and manage website content.
Customer	Registered users can browse available cars, create bookings, view booking history, make payments, and submit reviews and ratings for rented vehicles.
Guest	Guests can access public pages such as car listings, blogs, and website information, but cannot create bookings or access protected features without registration/login.

10. GUI Mockups

Deployment

a) Login Page

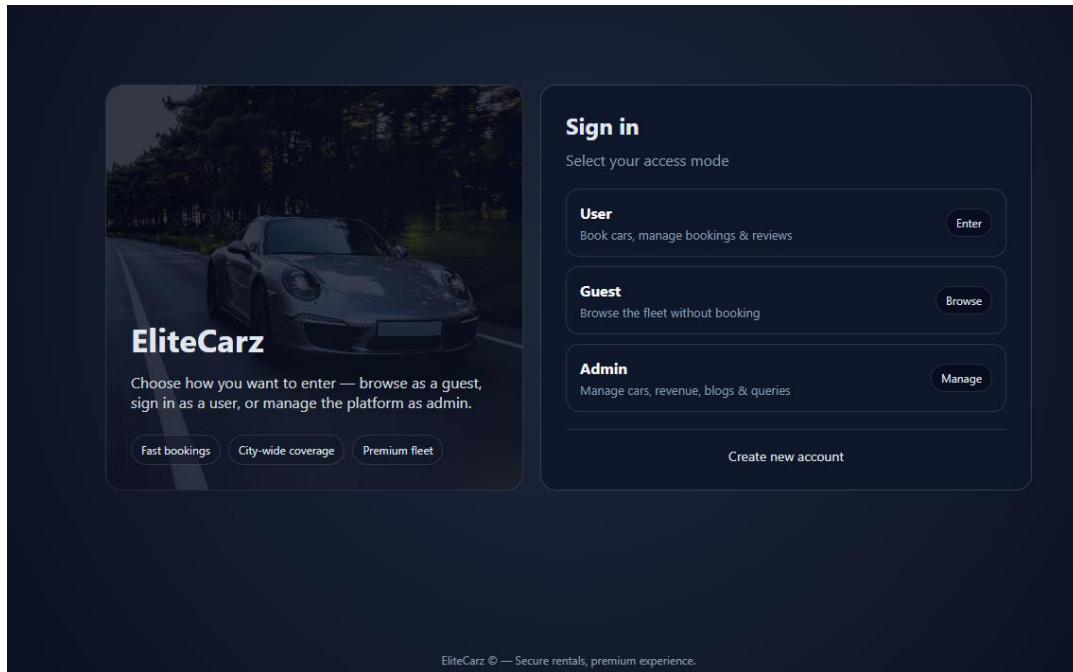


Figure 1 : Login Page

Role-base Access Control :

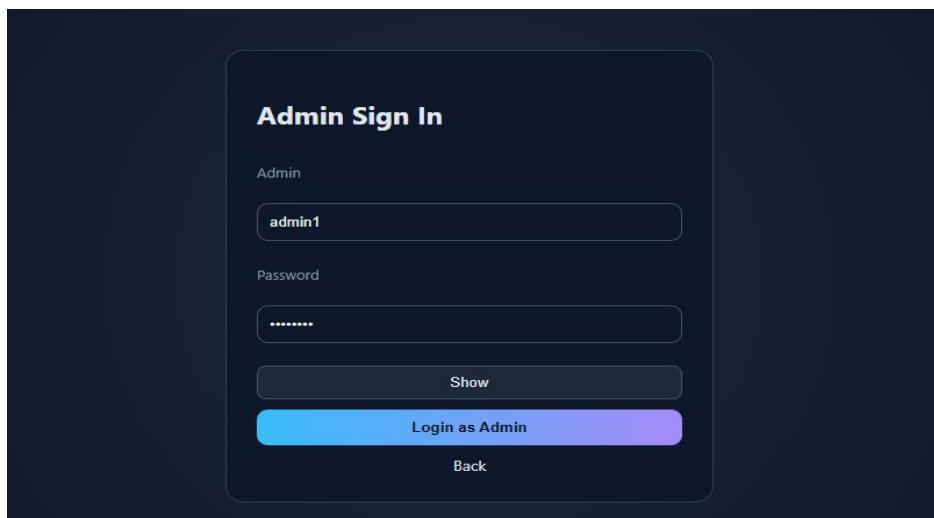
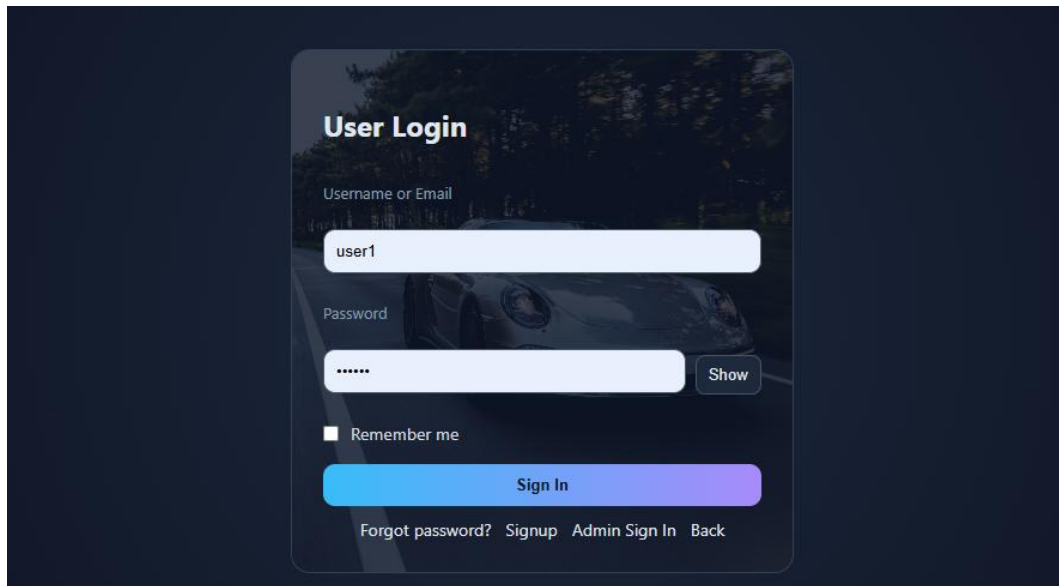


Figure 2 : Admin Login



The image shows a 'User Login' form on a dark background. The form has a title 'User Login' at the top. Below it are two input fields: 'Username or Email' with the value 'user1' and 'Password' with masked characters '*****'. There is a 'Show' button next to the password field. Below the password field is a checkbox labeled 'Remember me'. At the bottom of the form is a large 'Sign In' button. Below the button are links for 'Forgot password?', 'Signup', 'Admin Sign In', and 'Back'.

Figure 3 : User Login

b) Admins View

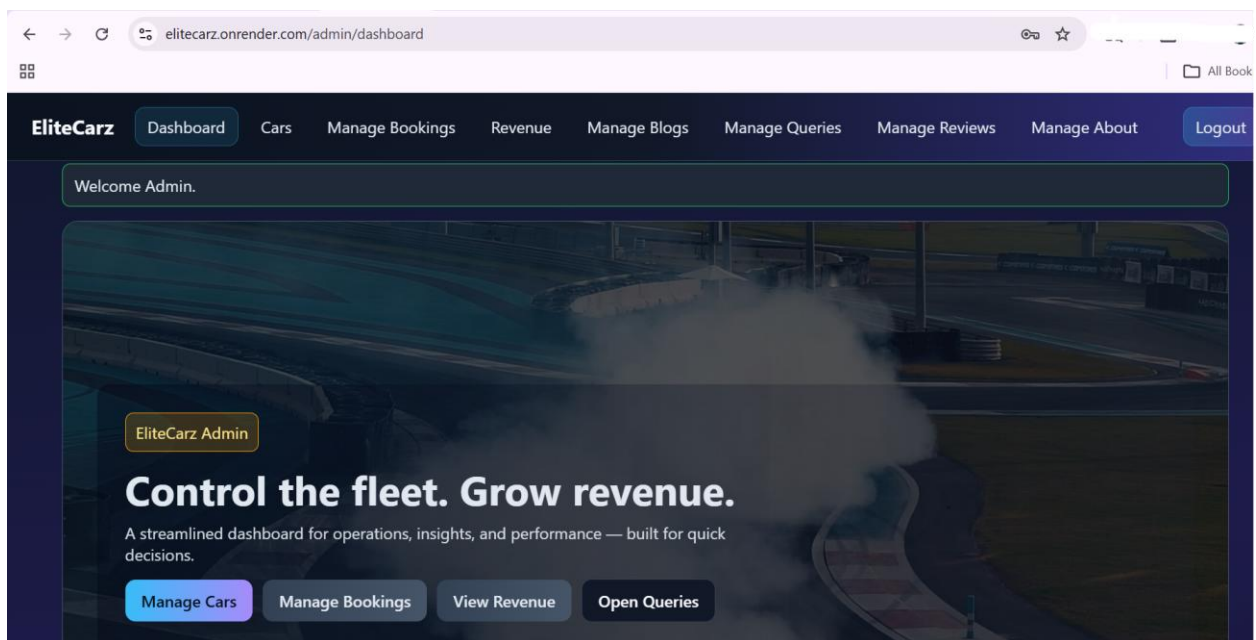


Figure A 1 : Admin Dashboard

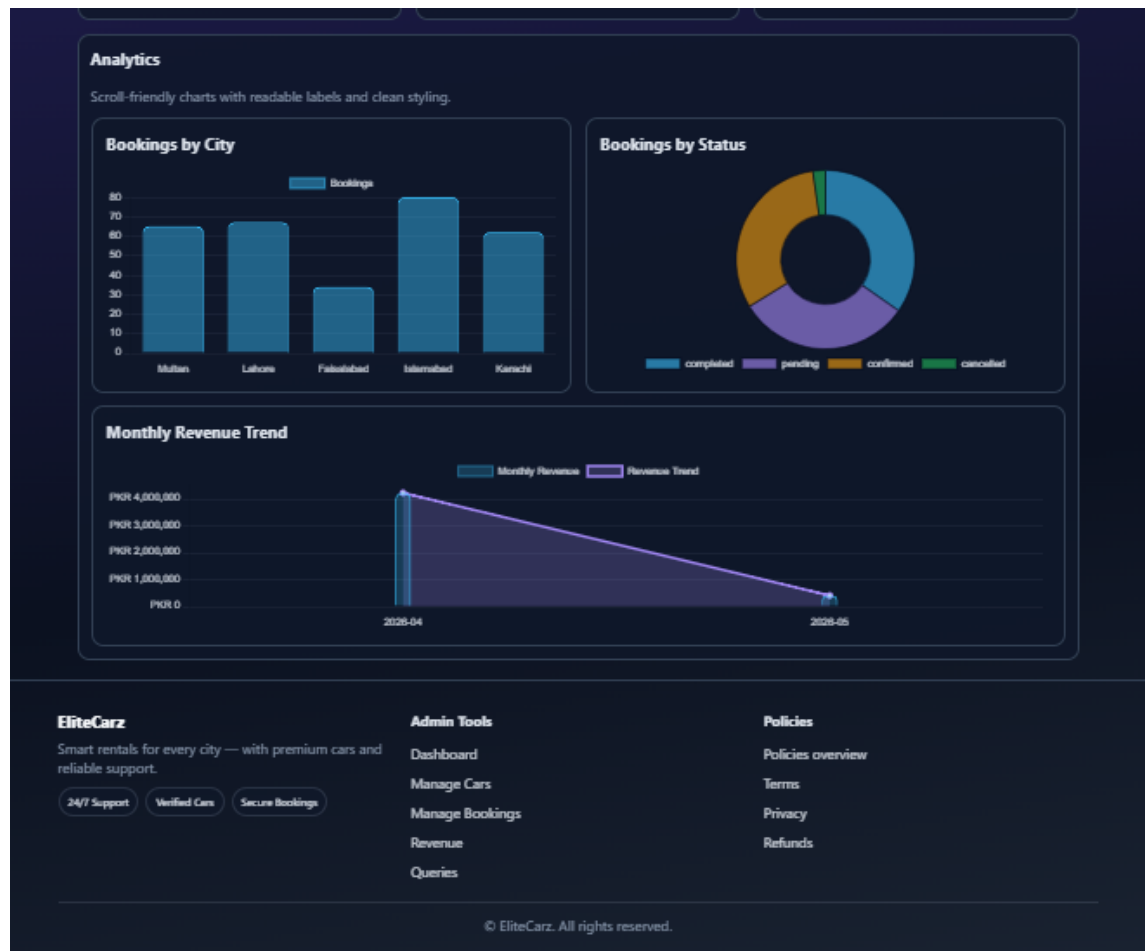


Figure A 2: Admin View Analytics

EliteCarz

Logout

Dashboard
Cars
Manage Bookings
Revenue
Manage Blogs
Manage Queries
Manage Reviews
Manage About

Manage Cars

Add new cars on demand, view the full fleet, and manage your monthly top performers.

Add Car

View All Cars

Showing: Top Cars of the Month

Search by ID, brand, model, city, category

Search by ID, brand, model, city, category

Search

Reset

Top Cars of the Month

Highest revenue performers for the current month. (This section hides when you press "View All Cars".)



CAR007 — Suzuki Swift

Multan • Revenue: PKR 167314 • Bookings: 1

Update Details



CAR004 — Toyota Land Cruiser

Karachi • Revenue: PKR 138726 • Bookings: 1

Update Details



CAR001 — BMW 3 Series

Islamabad • Revenue: PKR 78430 • Bookings: 2

Update Details



CAR036 — BMW 3 Series

Lahore • Revenue: PKR 11781 • Bookings: 1

Update Details

EliteCarz

Smart rentals for every city — with premium cars and reliable support.

24/7 Support
Verified Cars
Secure Bookings

Admin Tools

Dashboard
Manage Cars
Manage Bookings
Revenue
Queries

Policies

Policies overview
Terms
Privacy
Refunds

© EliteCarz. All rights reserved.

Figure A 3 : Admin Cars Page

EliteCarz

[Dashboard](#)
[Cars](#)
[Manage Bookings](#)
[Revenue](#)
[Manage Blogs](#)
[Manage Queries](#)
[Manage Reviews](#)
[Manage About](#)

Logout

Operations

Manage Bookings

Review every booking by date, inspect customer and car details, and move pending reservations to completed or cancelled.

Search booking id, client, car id, car name, city, status

Search booking id, client, car id, car name, city, sta

Search

Reset Filter

All bookings 399

Pending 194

Cancelled 8

Completed 107



BKG0201

BKG0201 - 2026-04-30 12:30:06.120473

Car: CAR100 - Mercedes C-Class

Client: User880

City: Karachi

Total: PKR 22677

View Details

Mark Completed

Cancel Booking



BKG0200

BKG0200 - 2026-04-30 12:30:06.120447

Car: CAR134 - Suzuki Swift

Client: User1878

City: Lahore

Total: PKR 30746

View Details

Mark Completed

Cancel Booking



BKG0199

BKG0199 - 2026-04-30 12:30:06.120421

Car: CAR056 - BMW 3 Series

Client: User513

City: Multan

Total: PKR 31723

View Details

Mark Completed

Cancel Booking



BKG0198

BKG0198 - 2026-04-30 12:30:06.120396

Car: CAR017 - Nissan Note

Client: User1976

City: Multan

Total: PKR 38751

View Details

Mark Completed

Cancel Booking



BKG0197

BKG0197 - 2026-04-30 12:30:06.120369

Car: CAR075 - BMW 3 Series

Client: User530

City: Multan

Total: PKR 6931

View Details

Mark Completed

Cancel Booking



BKG0196

BKG0196 - 2026-04-30 12:30:06.120341

Car: CAR096 - Mercedes C-Class

Client: User1034

City: Islamabad

Total: PKR 17609

View Details

Mark Completed

Cancel Booking



BKG0195

BKG0195 - 2026-04-30 12:30:06.120311

Car: CAR144 - Suzuki Cultus

Client: User2209

City: Faisalabad

Total: PKR 14004

View Details



BKG0194

BKG0194 - 2026-04-30 12:30:06.120286

Car: CAR096 - Mercedes C-Class

Client: User830

City: Islamabad

Total: PKR 35307

View Details

Mark Completed

Cancel Booking



BKG0193

BKG0193 - 2026-04-30 12:30:06.120258

Car: CAR144 - Suzuki Cultus

Client: User923

City: Faisalabad

Total: PKR 30201

View Details

Prev

Page 13 / 35

Next

EliteCarz

Smart rentals for every city — with premium cars and reliable support.

24/7 Support

Verified Cars

Secure Bookings

Admin Tools

[Dashboard](#)
[Manage Cars](#)
[Manage Bookings](#)
[Revenue](#)
[Queries](#)

Policies

[Policies overview](#)
[Terms](#)
[Privacy](#)
[Refunds](#)

© EliteCarz. All rights reserved.

Figure A 4 : Admin Booking Management Page

Page 19 | 45

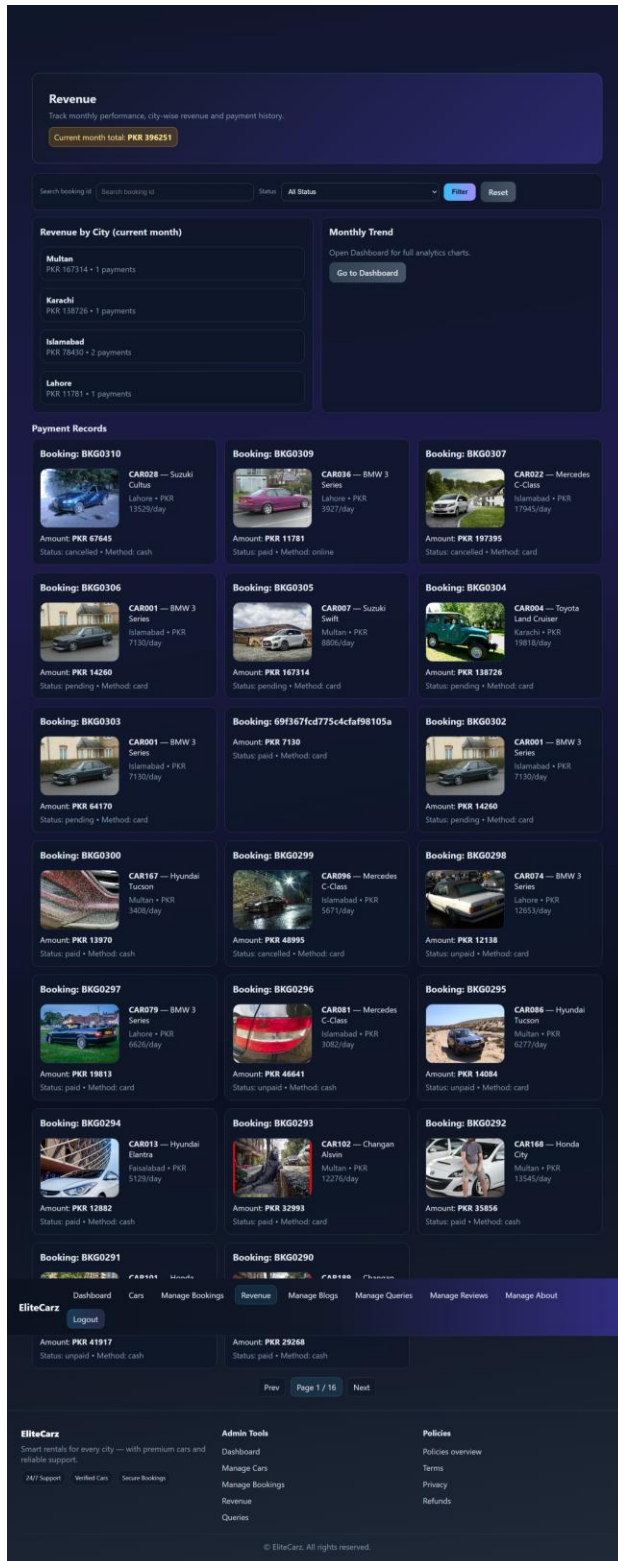


Figure A 5 : Admin Payment and Revenue Page

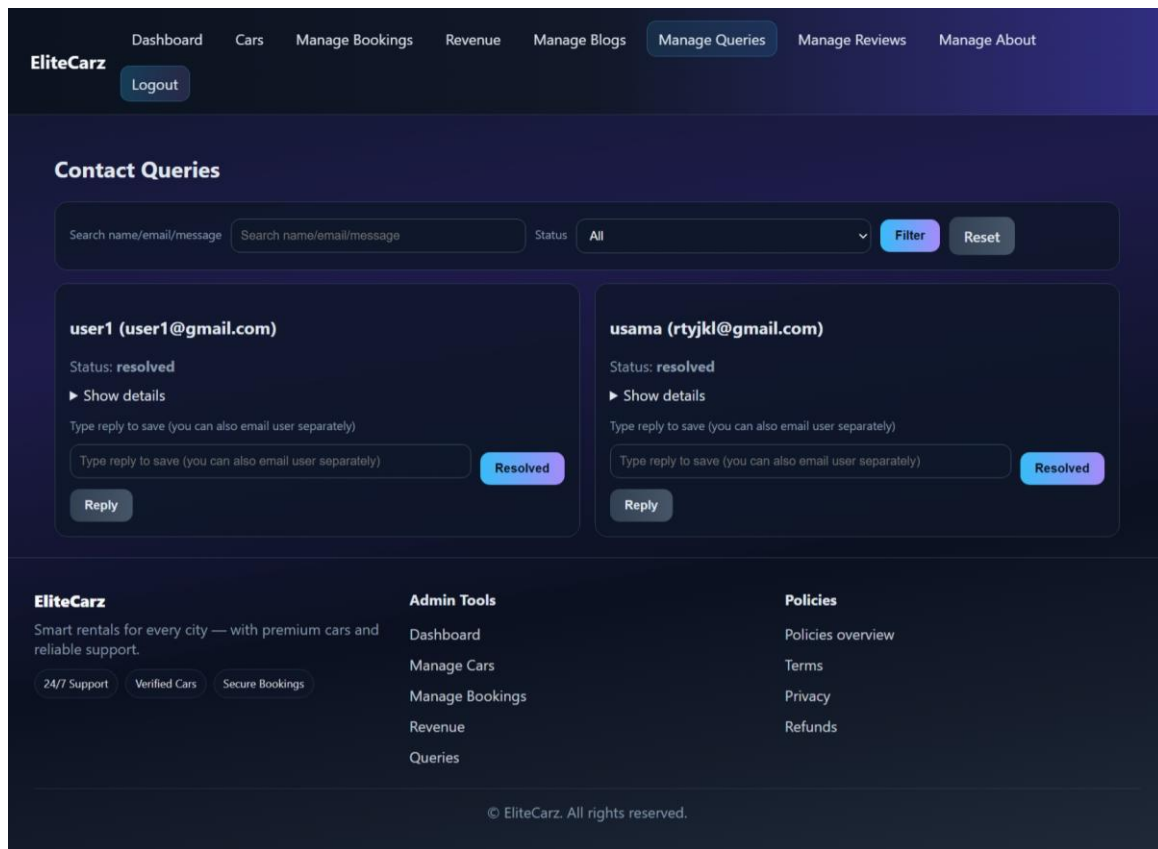


Figure A 6 : Admin Query management Page

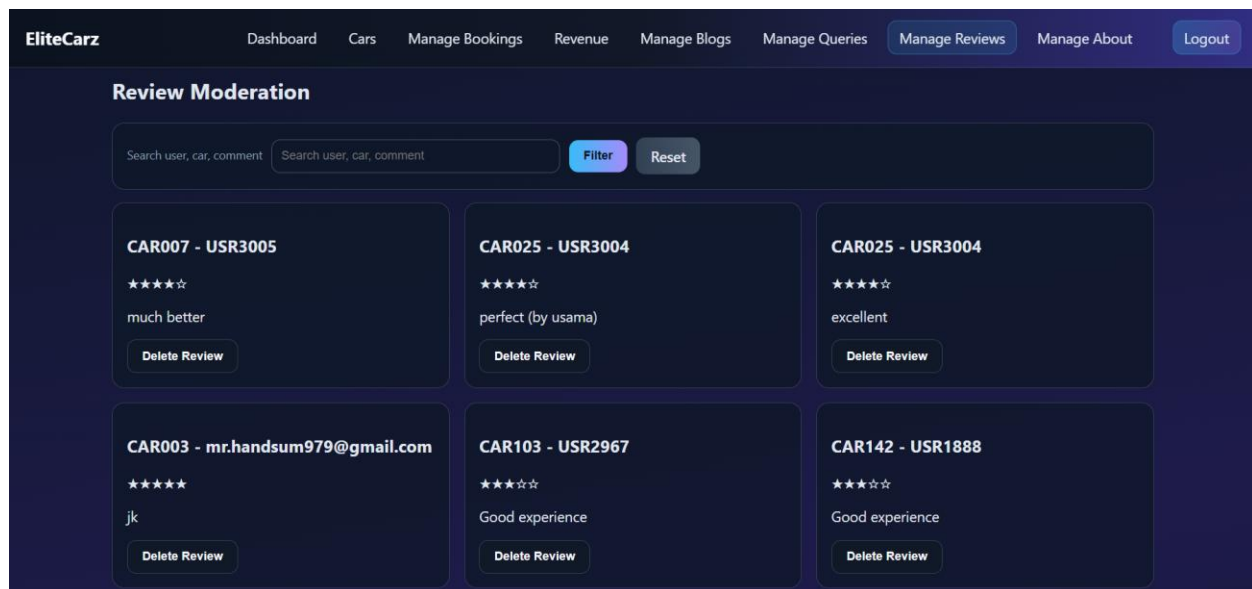


Figure A 7 : Admin Review management Page

c) Customer View

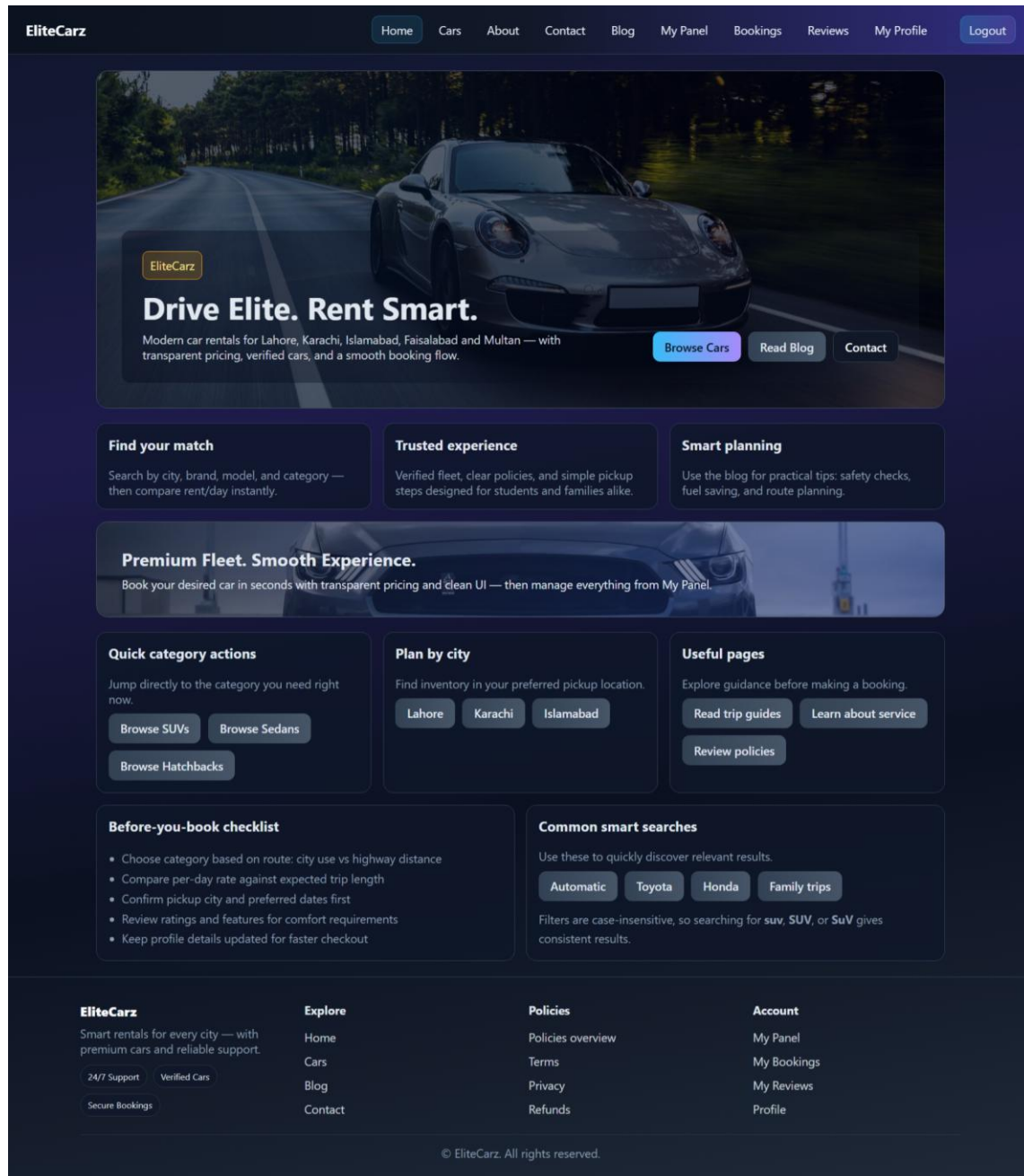


Figure C 1 : User Dashboard

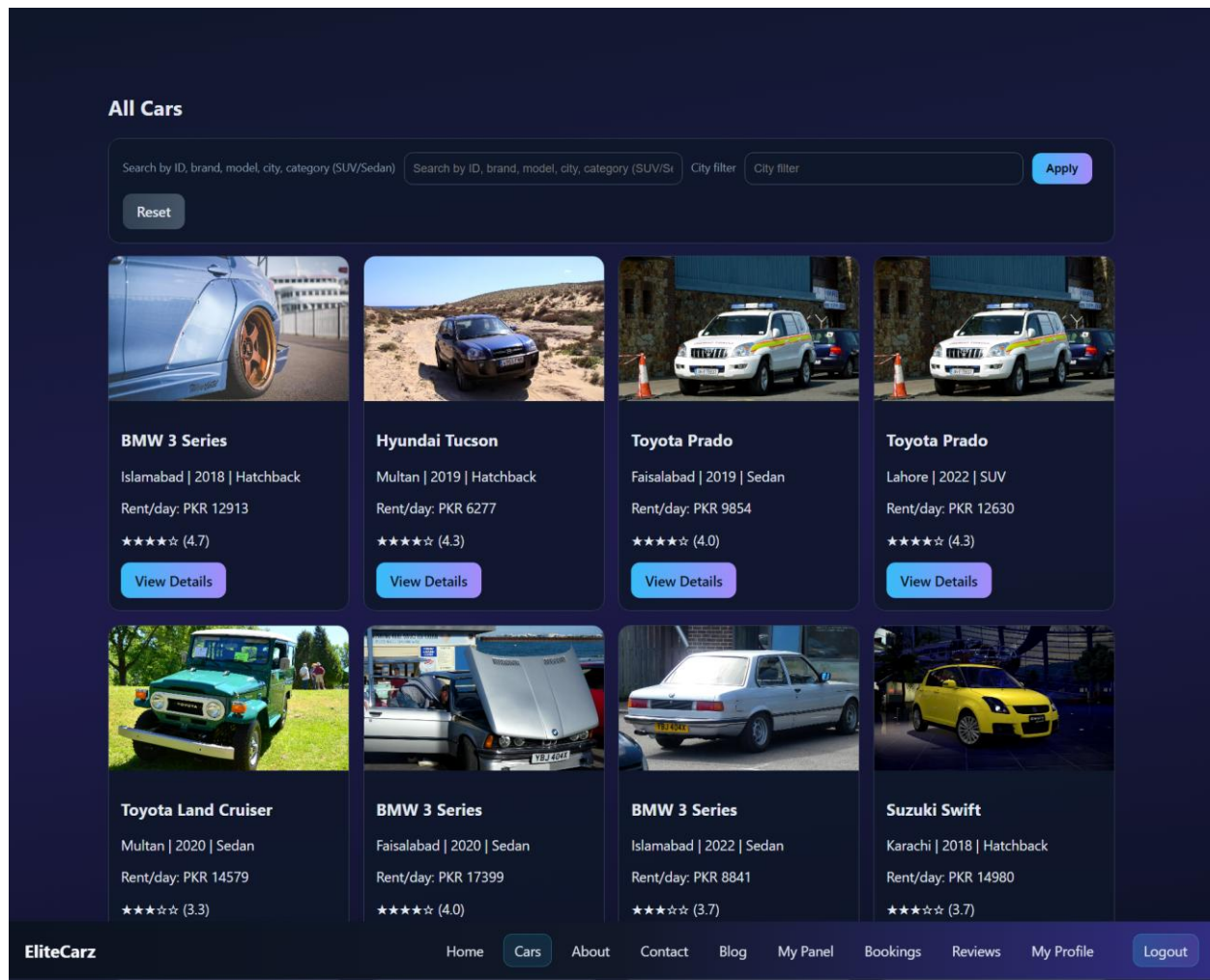


Figure C 2 : User Car Listings

EliteCarz

[Home](#)
[Cars](#)
[About](#)
[Contact](#)
[Blog](#)
[My Panel](#)
[Bookings](#)
[Reviews](#)
[My Profile](#)
[Logout](#)

[Filter](#)
[Reset](#)



ELC-BKG0310

Car: CAR028 — Suzuki Cultus

Status: Cancelled by EliteCarz (admin)

Reason: Cancelled by EliteCarz admin

Dates: 2026-05-19 00:00:00 to 2026-05-23 00:00:00

Total: PKR 67645

Pickup city: Lahore



ELC-BKG0309

Car: CAR036 — BMW 3 Series

Status: Completed

Dates: 2026-05-23 00:00:00 to 2026-05-25 00:00:00

Total: PKR 11781

Pickup city: Lahore



ELC-BKG0307

Car: CAR022 — Mercedes C-Class

Status: Cancelled by EliteCarz (admin)

Reason: Cancelled by EliteCarz admin

Dates: 2026-05-10 00:00:00 to 2026-05-20 00:00:00

Total: PKR 197395

Pickup city: Islamabad



ELC-BKG0306

Car: CAR001 — BMW 3 Series

Status: Auto cancelled

Reason: Auto-cancelled after 2 hours

Dates: 2026-05-09 00:00:00 to 2026-05-10 00:00:00

Total: PKR 14260

Pickup city: Islamabad



ELC-BKG0305

Car: CAR007 — Suzuki Swift

Status: Auto cancelled

Reason: Auto-cancelled after 2 hours

Dates: 2026-05-12 00:00:00 to 2026-05-30 00:00:00

Total: PKR 167314

Pickup city: Multan

EliteCarz
 Smart rentals for every city — with premium cars and reliable support.

[24/7 Support](#)
[Verified Cars](#)

[Secure Bookings](#)

Explore
[Home](#)
[Cars](#)
[Blog](#)
[Contact](#)

Policies
[Policies overview](#)
[Terms](#)
[Privacy](#)
[Refunds](#)

Account
[My Panel](#)
[My Bookings](#)
[My Reviews](#)
[Profile](#)

© EliteCarz. All rights reserved.

Figure C 3 : User Booking History

My Profile
View and update your details. For security, changes require your old password.

Profile details

Full name
user1

Username
user1

Change password

New password
New password
New password Show

Confirm new password
Confirm new password

Old password (required)
Old password
Old password Show

Update password

Tip: use at least 8 characters with a mix of letters and numbers.

EliteCarz Home Cars About Contact Blog My Panel Bookings Reviews My Profile Logout

Address
qwertyuuadff

Old password (required)
Old password
Old password Show

Save changes

EliteCarz
Smart rentals for every city — with premium cars and reliable support.
24/7 Support Verified Cars Secure Bookings

Explore
Home Cars Blog Contact

Policies
Policies overview Terms Privacy Refunds

Account
My Panel My Bookings My Reviews Profile

© EliteCarz. All rights reserved.

Figure C 4 : User Profile

11. Dataset Schema (MongoDB Collections)

The EliteCarz platform uses MongoDB Atlas as its primary NoSQL database. The database is designed using multiple collections to manage cars, bookings, payments, users, reviews, blogs, and analytical reports. Each collection is defined using JSON Schema Validation, that enforces flexible document-oriented structure , and allows efficient handling of semi-structured rental data while supporting scalability and performance.

Below are the main collections used in the EliteCarz database along with their schema structures:

11.1 User Collection

```
{  
  "_id": "String"  
  "name": "String"  
  "email": "String"  
  "password": "String"  
  "role": "String"  
  "created_at": "Date"  
}
```

11.2 Cars Collection

```
{  
  "_id": "String"  
  "brand": "String"  
  "model": "String"  
  "year": "Number"  
  "category": "String"  
  "price_per_day": "Number"  
  "city": "String"  
  "features": "Array<String>"  
  "ratings": "Array<Number>"  
  "availability": "Boolean"  
  "version": "Number"  
  "image_url": "String"  
  "created_at": "Date"  
}
```

11.3 Booking Collection

```
{  
  "_id": "String"  
  "user_id": "String"  
  "car_id": "String"  
  "start_date": "Date"
```

CSC 316 – Advanced Database Systems

```
"end_date": "Date"
"status": "String"
"payment_status": "String"
"total_price": "Number"
"version": "Number"
"created_at": "Date"}
```

11.4 Payment Collection

```
{
  "_id": "String"
  "booking_id": "String"
  "amount": "Number"
  "status": "String"
  "method": "String"
  "created_at": "Date"
}
```

11.5 Reviews Collection

```
{
  "_id": "String"
  "user_id": "String"
  "car_id": "String"
  "rating": "Number"
  "comment": "String"
  "created_at": "Date"
}
```

11.6 Blog Post Collection

```
{
  "_id": "String"
```

CSC 316 – Advanced Database Systems

```
"title": "String"
"content": "String"
"author": "String"
"created_at": "Date" }
```

11.7 Contact Queries Collection

```
{
  "_id": "ObjectId"
  "name": "String"
  "email": "String"
  "message": "String"
  "status": "String"
  "admin_reply": "String"
  "created_at": "Date"
  "replied_at": "Date"
}
```

11.8 Site Content Collection

```
{
  "_id": "ObjectId"
  "key": "String"
  "content": "String"
  "updated_at": "Date"
}
```

11.9 Counter Collection

```
{
  "_id": "String"
  "seq": "Number"
}
```

```
}
```

Purpose

- Simulate auto-increment functionality in MongoDB
- Generate sequential booking IDs

11.10 Locks Collection

This collection is used for pessimistic locking and concurrency control.

```
{  
  "_id": "String"  
  "resource": "String"  
  "locked_at": "Date"  
}
```

11.11 Booking Payment (View Only)

This MongoDB View combines booking and payment information for reporting purposes.

Purpose

- Simplify booking-payment analytics
- Display combined booking and payment records

11.12 User_Booking_view (MongoDB View)

This view combines user and booking information.

Purpose

- Provide user-wise booking reports
- Simplify dashboard analytics
- Improve reporting readability

11.13 mv_monthly_revenue (Materialized View)

This materialized view stores monthly revenue summaries generated through aggregation pipelines.

```
{
  "_id": "String",
  "total": number
}
```

11.14 mv_revenue_by_city_current_month (Materialized View)

This materialized view stores city-wise revenue analytics for the current month.

Purpose

- Analyze city-based rental performance
- Generate revenue comparison reports

11.15 mv_top_cars_current_month (Materialized View)

This materialized view stores top-performing cars based on bookings and revenue.

```
{
  "_id": "String"
  "total": "Number"
  "bookings": "Number"
  "car": "Object" }
```

12. Improvements made in Dataset Schema

a) Improvements Made in the Dataset Schema

During the transition from the **Midterm Project Schema** to the **Final Project Schema**, significant improvements were made to support the new requirements of the Car Rental Management System.

These enhancements include better normalization, richer user structures, versioning for concurrency control, improved rating and subscription mechanics, the addition of new collections, and schema validation upgrades.

This section explains **what improvements were made**, and **why they were necessary**.

b) Summary of Improvements

i. Users Collection

Old Schema (Customers)

- name
- email
- phone
- license_no

New Schema (Users)

- added password (authentication)
- added role (user/admin access control)
- added created_at

Improvement: Secure login system + role-based access

ii. Cars Collection

Old Schema

- model
- brand
- price_per_day
- availability
- location_id
- type_id

New Schema

- added year, city, features, image_url
- added ratings array
- added version field

CSC 316 – Advanced Database Systems

Improvement: Better data detail + optimistic locking + UI support

iii. Bookings Collection

Old Schema

- customer_id
- car_id
- start_date
- end_date
- status
- total_amount

New Schema

- added payment_status
- added version
- improved status lifecycle (pending/confirmed/completed/cancelled)

Improvement: Full booking lifecycle + concurrency control

iv. Payments Collection

Old Schema

- booking_id
- amount
- method
- status
- paid_date

New Schema

- added structured status (paid/pending/refunded)
- improved method handling (cash/card/wallet)
- timestamps improved

Improvement: Better financial tracking system

v. Reviews Collection

Old Schema

- customer_id
- car_id
- rating
- comment
- date

New Schema

- improved relation with users and cars
- standardized rating system
- added timestamps

Improvement: Better feedback accuracy + analytics support

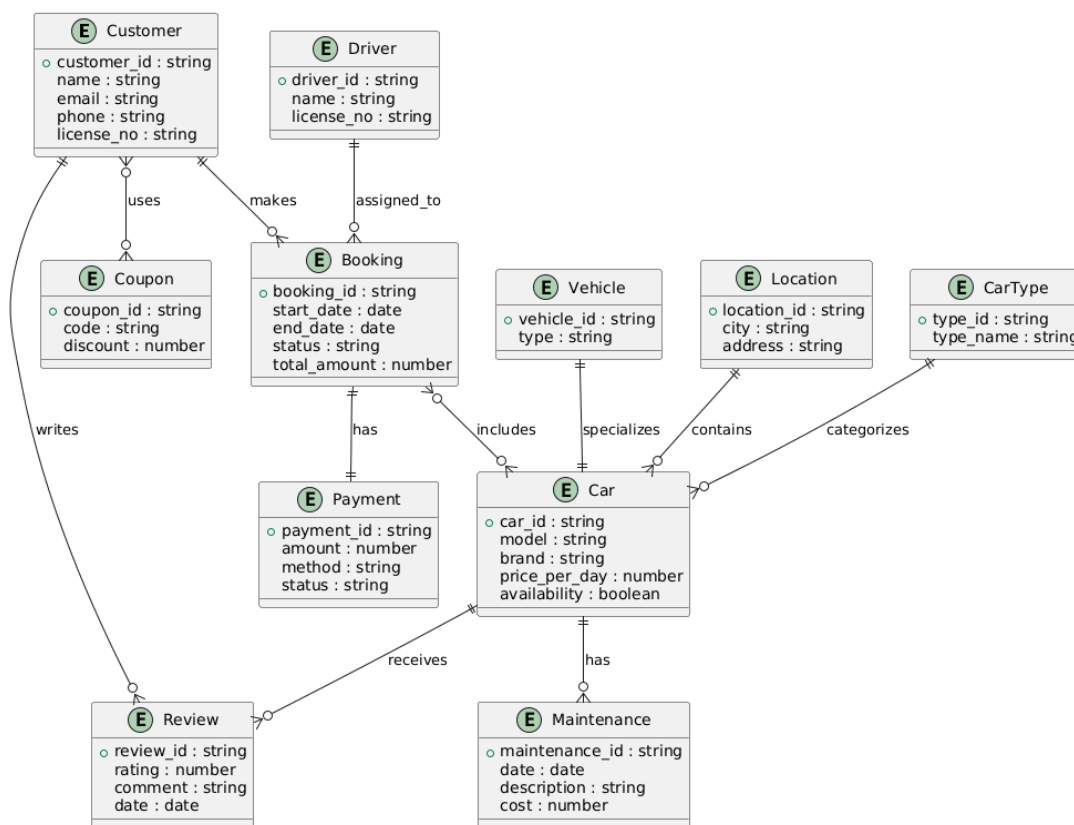
c) Addition of Multiple New Collections

Collection	Purpose
Blog Posts	Added to introduce a CMS (Content Management System). It allows admins to publish blogs for user, supports dynamic content, improving user engagement.
Contact Queries	Added to handle customer support messages and admin replies. This improves communication flow and makes the system more user-friendly
Site Content	Added for dynamic page management (About, Terms, etc.). It allows admins to update website content without code changes, making the system more flexible
Counters	Added to generate sequential IDs for records like bookings and improves data consistency and scalability.
Locks	Added for pessimistic locking to prevent concurrent booking issues

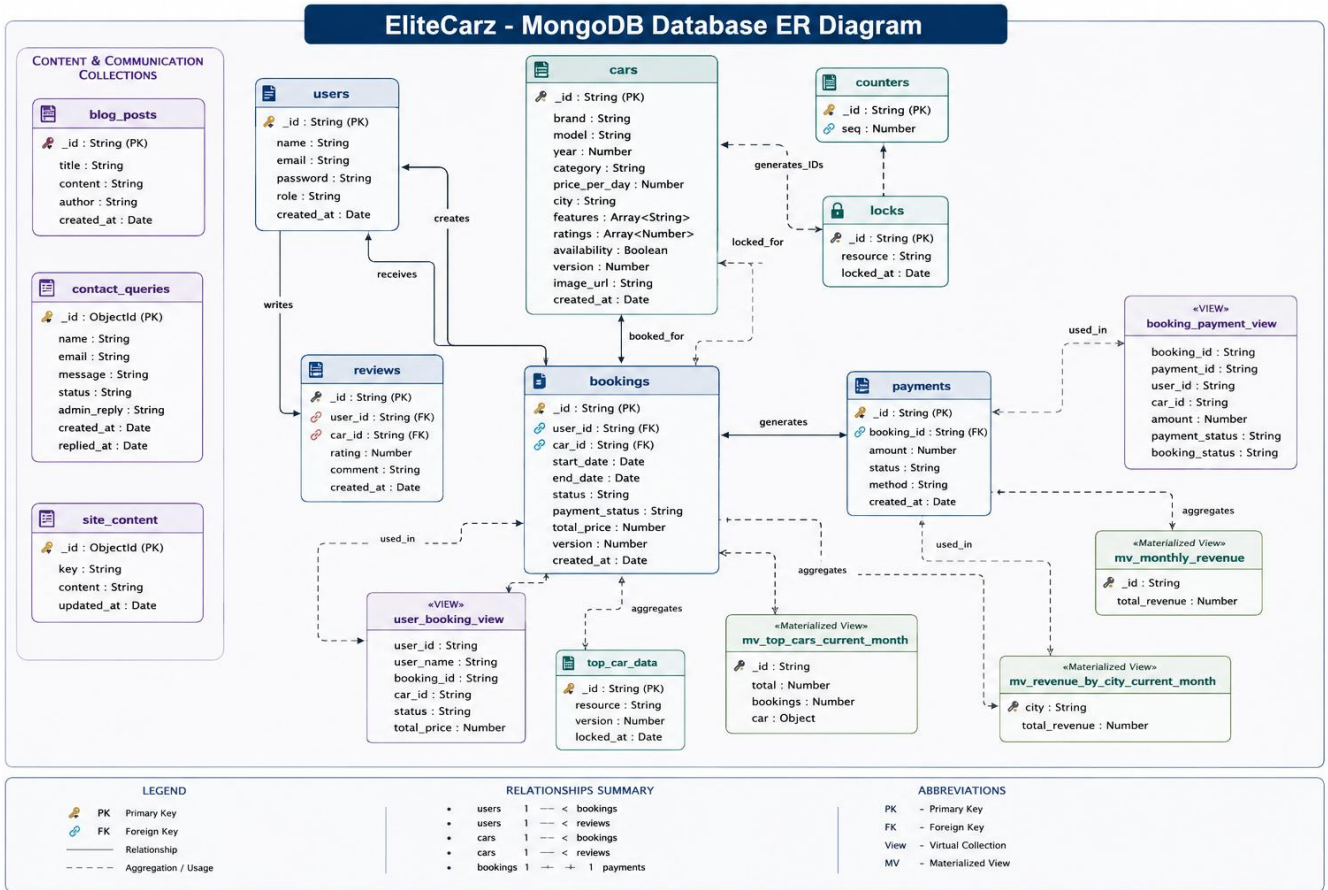
Why These Improvements are Needed

- Concurrency control and locking are added to prevent double booking and ensure data consistency.
- Authentication with roles is added to secure the system and separate admin/user access.
- CMS-related collections are added to make the system dynamic instead of static.
- Counters are added to handle auto-increment IDs and improve data consistency.
- Views and materialized views are added to improve reporting and performance.

13. ER Diagrams



ERD 1 : ERD for MIDS Project



ERD 2 : ERD for FINAL Project

14. Lab Implementations (Major Section)

The EliteCarz system demonstrates the practical implementation of Advanced Database Systems concepts using MongoDB Atlas. The system integrates CRUD operations, aggregation pipelines, transactions, concurrency control, locking mechanisms, schema validation, and views to simulate a real-world car rental platform.

14.1 CRUD Operations (LAB 1–2)

The EliteCarz platform implements full CRUD (Create, Read, Update, Delete) operations across multiple collections such as cars, users, bookings, reviews, and payments.

Create Operations

CSC 316 – Advanced Database Systems

```
db.cars.insert_one({
  "_id": "CAR999",
  "brand": "Toyota",
  "model": "Corolla",
  "year": 2022,
  "price_per_day": 8500,
  "city": "Lahore",
  "availability": True,
  "version": 1
})
```

Read Operations

```
db.cars.find({"city": "Islamabad"})
```

Update Operations

```
db.cars.update_one(
  {"_id": "CAR001"},
  {"$set": {"availability": False}}
)
```

Delete Operations

```
db.cars.delete_one({"_id": "CAR001"})
```

Booking CRUD Example

```
db.bookings.insert_one({
  "_id": "BKG1001",
  "user_id": "USR0009",
  "car_id": "CAR001",
  "status": "pending",
  "payment_status": "unpaid"
})
```

14.2 Aggregation Pipelines (LAB 3–4)

Aggregation pipelines are used in EliteCarz for generating analytics such as revenue reports, top cars, and booking statistics.

Monthly Revenue Calculation

```
db.bookings.aggregate([
  {
    "$group": {
      "_id": {"$substr": ["$created_at", 0, 7]},
```

```
        "total_revenue": {"$sum": "$total_price"}
    }
}
])
```

Top Cars Based on Bookings

```
db.bookings.aggregate([
  {
    "$group": {
      "_id": "$car_id",
      "total_bookings": {"$sum": 1},
      "revenue": {"$sum": "$total_price"}
    }
  },
  {"$sort": {"revenue": -1}},
  {"$limit": 5}
])
```

Revenue by City

```
db.bookings.aggregate([
  {
    "$lookup": {
      "from": "cars",
      "localField": "car_id",
      "foreignField": "_id",
      "as": "car_info"
    }
  },
  {"$unwind": "$car_info"},
  {"$group": {
    "_id": "$car_info.city",
    "total_revenue": {"$sum": "$total_price"}
  }}
])
```

14.3 Data Consistency (Versioning + Atomic Updates)

EliteCarz ensures data consistency using version-based optimistic locking in bookings and cars collections.

Where Implemented

- Movies collection uses a `version` field
- Ratings update uses **atomic operations** (`$inc`, `$set`)
- Prevents lost updates in concurrent writes

Optimistic Locking Example

```
db.bookings.update_one(  
  {  
    "_id": "BKG0009",  
    "version": 1  
  },  
  {  
    "$set": {"status": "confirmed"},  
    "$inc": {"version": 1}  
  }  
)
```

Atomic Update for Cars

```
db.cars.update_one(  
  {"_id": "CAR001"},  
  {"$set": {"availability": False}}  
)
```

Purpose

- Prevents lost updates
- Ensures data integrity during concurrent access

14.4 Transactions (Atomic Multi-Document Operations)

Transactions are used to ensure booking and payment consistency.

Booking + Payment Transaction

```
with client.start_session() as session:  
    with session.start_transaction():
```

```
db.bookings.insert_one({
    "_id": "BKG2001",
    "user_id": "USR0009",
    "car_id": "CAR001",
    "status": "confirmed"
}, session=session)

db.payments.insert_one({
    "_id": "PAY2001",
    "booking_id": "BKG2001",
    "amount": 30000,
    "status": "paid"
}, session=session)
```

Purpose

- Ensures atomicity
- Either both operations succeed or fail together

14.5 Concurrency Control

EliteCarz handles concurrent bookings using versioning and locking mechanisms.

Supporting Techniques

- Optimistic Locking (via `version` fields)
- Demonstration of Pessimistic Locking (via `locks` collection)

Relevant Code Snippet

➤ Optimistic Locking (Car availability Update)

```
db.cars.update_one(
    {"_id": "CAR001", "version": 1},
    {
        "$set": {
            "availability": False
        },
        "$inc": {
            "version": 1
        }
    })
```

➤ Pessimistic Locking (`locks` Collection)

```
db.locks.insert_one({
```

```
    "_id": "LOCK001",  
    "resource": "CAR001",  
    "locked_at": "2026-05-01T12:00:00Z"  
  })
```

14.6 Locking Types Used

Relevant Code Snippet

```
db.locks.insert_one({  
  "_id": "LOCK001",  
  "resource": "CAR001",  
  "locked_at": "2026-05-01T12:00:00Z"  
})
```

Lock Type	Where Used
Document-Level Locking	MongoDB internal engine during writes
Optimistic Locking	Cars and Bookings <code>version</code> field
Pessimistic Locking	<code>locks</code> collection
Application-Level Locks	Manual lock entries before update

14.7 Views and Materialized Views

EliteCarz uses MongoDB Views and Materialized Views for analytics optimization.

MongoDB View (`booking_payment_view`)

```
db.createView(  
  "booking_payment_view",  
  "payments",  
  [ {  
    $lookup: {  
      from: "bookings",  
      localField: "booking_id",  
      foreignField: "_id",  
      as: "booking_details"  
    }  
  } ] )
```

User Booking View

```
db.createView(  
  "user_booking_view",  
  "bookings",  
  [ {  
    $lookup: {  
      from: "users",  
      localField: "user_id",  
      foreignField: "_id",  
      as: "user_info"  
    }  
  } ] )
```

Materialized View (Code Snippet)

```
db.bookings.aggregate([  
  {  
    "$group": {  
      "_id": {"$substr": ["$created_at", 0, 7]},  
      "total": {"$sum": "$total_price"}  
    } },  
  {  
    "$out": "mv_monthly_revenue"  
  }  
])
```

14.8 Database Security (Authentication + Access Control)

EliteCarz uses role-based authentication and password hashing for security.

Features Implemented :

- Password hashing (salted)
- Protected admin routes
- Role-based navigation (Admin/User/Guest)

Hashing code snippet

```
hashed_password = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
```

RBAC Enforcement

CSC 316 – Advanced Database Systems

```
if user.role != "admin":  
    return {"error": "Access Denied"}
```

Purpose

- Protects sensitive data
- Ensures authorized access only

14.9 NoSQL Injection Prevention

EliteCarz prevents NoSQL injection by avoiding raw query execution.

How Prevented

- Queries always use dictionary structures
- User input **never** inserted as operators
- Regex used safely with `$options: "i"`
- No `$where`, no string-based query construction

Relevant Code Snippet

```
db.cars.find({  
  "city": {"$regex": "Lahore", "$options": "i"}  
})
```

14.10 Salted Password Hashing

Where Implemented:

- Registration
- Login verification

Relevant Code snippet

Passwords are stored securely using hashing techniques.

```
bcrypt.hashpw(password.encode(), bcrypt.gensalt())  
  
db.users.insertOne({"password" : hashed_password
```

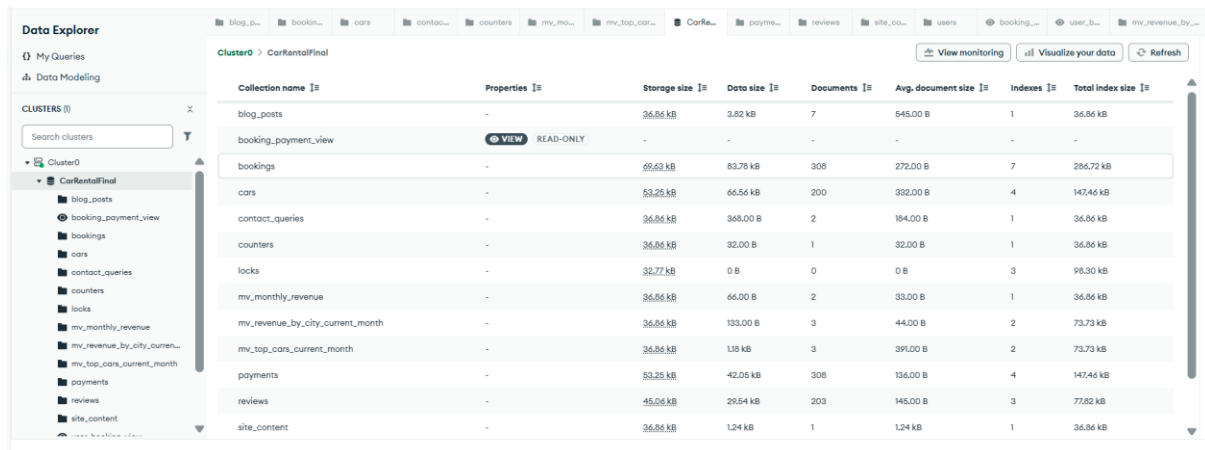


14.11 Access Control (Admin/User/Guest)

Role	Permissions
Admin	Full CRUD + Analytics
User	Booking, reviews, payments
Guest	Browse cars and blogsonly

14.12 Use of MongoDB Atlas

EliteCarz uses MongoDB Atlas cloud database for scalability and reliability.



The screenshot shows the MongoDB Atlas Data Explorer interface. On the left, there's a sidebar with 'Data Explorer' and 'My Queries'. The main area displays a table of collections for a cluster named 'CarRentallFinal'. The table includes columns for Collection name, Properties, Storage size, Data size, Documents, Avg. document size, Indexes, and Total index size. The collections listed are: blog_posts, booking_payment_view, bookings, cars, contact_queries, counters, locks, mv_monthly_revenue, mv_revenue_by_city_current_month, mv_top_cars_current_month, payments, reviews, and site_content.

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
blog_posts	-	36.86 kB	3.82 kB	7	545.00 B	1	36.86 kB
booking_payment_view	VIEW READ-ONLY	-	-	-	-	-	-
bookings	-	69.63 kB	83.79 kB	308	272.00 B	7	286.72 kB
cars	-	53.25 kB	66.56 kB	200	332.00 B	4	147.46 kB
contact_queries	-	36.86 kB	368.00 B	2	184.00 B	1	36.86 kB
counters	-	36.86 kB	32.00 B	1	32.00 B	1	36.86 kB
locks	-	32.77 kB	0 B	0	0 B	3	98.30 kB
mv_monthly_revenue	-	36.86 kB	66.00 B	2	33.00 B	1	36.86 kB
mv_revenue_by_city_current_month	-	36.86 kB	133.00 B	3	44.00 B	2	73.73 kB
mv_top_cars_current_month	-	36.86 kB	118 kB	3	391.00 B	2	73.73 kB
payments	-	53.25 kB	42.05 kB	308	136.00 B	4	147.46 kB
reviews	-	46.04 kB	29.54 kB	203	145.00 B	3	77.82 kB
site_content	-	36.86 kB	1.24 kB	1	1.24 kB	1	36.86 kB

Advantages

- Cloud-based storage
- High availability
- Supports transactions
- Real-time scalability

15. System Deployment

The **EliteCarz** web application has been successfully **deployed on Render** (a cloud hosting platform), making it accessible over the internet. The deployment ensures that users can interact with the system in real-time without requiring local setup.

The system architecture follows a client-server model where the frontend communicates with the backend API, and MongoDB Atlas is used as the cloud database.

Deployment Features:

CSC 316 – Advanced Database Systems

- Fully cloud-hosted web application
- Real-time database connectivity using MongoDB Atlas
- Secure environment variable configuration
- Public accessibility for users and administrators
- Scalable architecture supporting multiple concurrent users

Benefits of Deployment:

- Eliminates the need for local installation
- Provides 24/7 availability
- Improves system reliability and performance
- Enables real-world usability testing

Live URL :

 Live Project URL: <https://elitecarz.onrender.com/>

16. Conclusion

The EliteCarz successfully demonstrates the practical implementation of Advanced Database Systems concepts using MongoDB Atlas and Flask. The project integrates CRUD operations, aggregation pipelines, transactions, concurrency control, schema validation, indexing, and security mechanisms into a real-world rental management application.

The system provides efficient vehicle booking management, analytics dashboards, payment tracking, and role-based access control while maintaining scalability and flexibility through MongoDB's document-oriented architecture. Advanced features such as optimistic locking, transactions, salted password hashing, and MongoDB views further strengthen the system's reliability and performance.

Overall, the project fulfills the objectives of the Advanced Database Systems course and demonstrates how NoSQL technologies can be applied to solve real-world business problems.

17. References

- **MongoDB Documentation:** <https://www.mongodb.com/docs/>
- **MongoDB Aggregation Pipeline Guide:**
<https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>

- **MongoDB Schema Validation:** <https://www.mongodb.com/docs/manual/core/schema-validation/>
- **Flask Framework Documentation:** <https://flask.palletsprojects.com/>
- **Flask Security Best Practices:** <https://flask.palletsprojects.com/en/latest/security/>
- **bcrypt Password Hashing Documentation:** <https://pypi.org/project/bcrypt/>
- **Chart.js Official Documentation:** <https://www.chartjs.org/docs/latest/>
- **Python MongoDB Driver (PyMongo) Documentation:** <https://pymongo.readthedocs.io/en/stable/>
- **MongoDB Atlas Documentation:** <https://www.mongodb.com/docs/atlas/>